Asincronismo

Daniel Sebastián Dueñas Vázquez
Carrera de Desarrollo de Software
Instituto Superior Tecnológico Internacional ITI
Materia: Programación de Aplicaciones Web y Móbiles

Docente: Ing. David Herdoíza, MsC.

1 de noviembre de 2026

Consulte sobre el asincronismo en JavaScript y responda las siguientes preguntas de manera crítica. Además, suba un ejemplo de cada uno al repositorio de GitHub.

- **¿Cuál es el funcionamiento de los callbacks?**

Una función callback asincrónica es una función que se pasa a otra como un argumento, que luego se ejecuta (o se "llama nuevamente") en un momento posterior para completar una tarea específica.

El problema es que depende de que la otra función la llame con los argumentos correctos, o sea pierdes control. También puede ocurrir que la función externa, al ejecutar el callback, llame dos veces a la función interna y altere los argumentos, no la llame o la llame muy pronto o tarde. El manejo de errores es más complicado, debido a que el bloque try/catch no logra capturar los errores de un callback asincrónico. Otro problema es que se pierde legibilidad, porque puedes llegar a tener muchas funciones anidadas y eso crea una estructura que es difícil de leer.

- **¿Cómo mejora el uso de promesas?**

Una promesa es un objeto que representa el resultado eventual (pending, fullfilled y rejected) de una operación asíncrona. Entonces, en vez de pasar tu lógica a la función, ella te devuelve un objeto sobre el que tú decides qué hacer cuando esté listo. Así, tú mantienes el control del flujo del código y no dependes de cómo la función maneje internamente tu lógica. El manejo de errores es mejor, porque usas un .catch al final de la cadena. La legibilidad también mejora, no se crea la estructura triangular como un las funciones callbacks anidadas.

- **¿Cómo se revolucionó el manejo asincrónico con async y await?**

A primera vista, lo que mejoró el async/await fue la legibilidad del código. Además puedes manejar todos los errores con el mismo bloque try/catch, al igual que si fuera síncrona, y esto es posible porque el async/await se construyen sobre promesas. La función async devuelve una promesa y el await pausa la función hasta que la promesa se resuelva. Así desaparece las estructura triangular y las cadenas de .then().

Aunque tiene algunos inconvenientes prácticos, como que se te olvide un await, entonces estas trabajando solo con una promesa pendiente. O, que tengas varios await seguidos para tareas independientes y se ejecuten uno tras otro sin necesidad.